Arquitectura con modelo C4

E5 · PROYECTO FINAL BACKEND

Kamá

Plataforma de autoaprendizaje digital para la sala de internet del CLENA

Equipo: Kamá · Soluciones para el CLENA

Integrantes: Wilmer, Esteban, Andrés y Carlos.
Nombres disponibles en la documentación del repositorio.

Fecha: 13 de noviembre de 2026
Versión: 1.0
Base de código inspeccionada: commit 07a7b05

Informe de arquitectura de la implementación local. Incluye contexto, contenedores, componentes de la API, estructura real del backend, decisiones técnicas e inyección de dependencias.

Índice y alcance

Contenido	Página
Portada	1
Índice y alcance	2
Nivel 1 — Contexto	3
Nivel 2 — Contenedores	4
Comunicaciones y despliegue	5
Nivel 3 — Componentes de la API	6
Clean Architecture y recorrido de una petición	7
Estructura real del backend	8
Decisiones técnicas justificadas	9
Composición NestJS e inyección por token	10
Trazabilidad y límites de la entrega	11

El informe responde al entregable E5 de *Entrega 1.pdf*, páginas 2–3: niveles C4 1 a 3, carpetas reales, decisiones justificadas y fragmento de composición NestJS. El nivel 4 es opcional y no se incluye.

Kamá atiende el autoaprendizaje digital mediante categorías, guías con pasos e imágenes, actividades de práctica y solicitudes de ayuda presencial. También incorpora identidad, sesiones, privacidad y administración del personal. La arquitectura describe responsabilidades; no certifica por sí sola que todas las historias cumplan la definición de terminado de E7.

Se toma el código como fuente del estado implementado. Las conexiones de correo/SMS existen como adaptadores configurables; el Compose actual utiliza los valores por defecto de notificaciones en consola. HTTPS se identifica como objetivo de despliegue, pues el entorno local usa HTTP.

1. Nivel 1 — Contexto

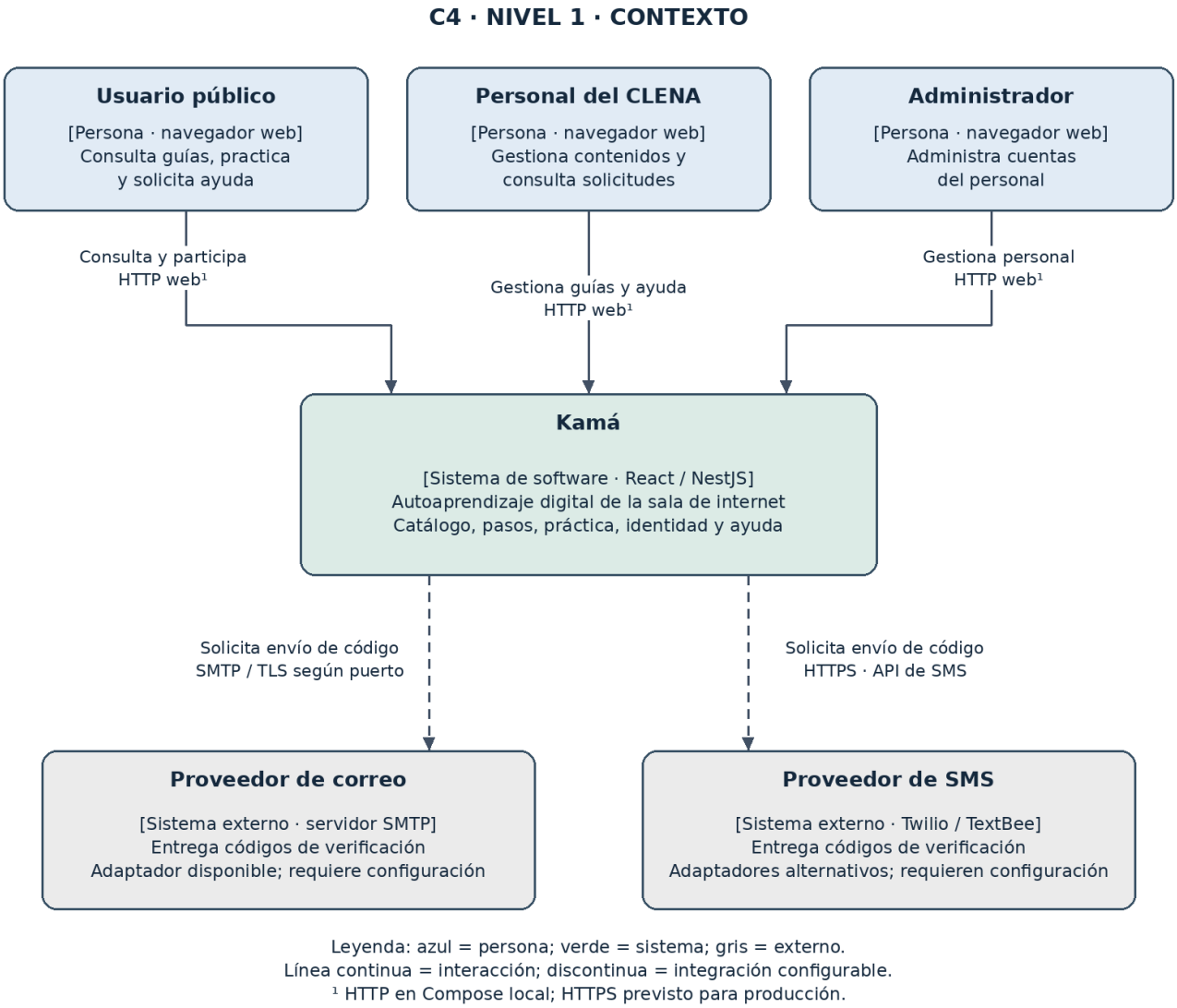


Figura 1. Kamá, actores y sistemas externos.

El usuario público puede explorar el catálogo sin cuenta y acceder a funciones de identidad y aprendizaje. El personal mantiene contenidos y consulta ayuda; el administrador gestiona cuentas del personal. El proveedor de correo y el de SMS permiten enviar códigos de verificación; no son servicios desarrollados dentro de Kamá.

No se incluye un simulador de sensores porque no corresponde al dominio de este proyecto. La interacción externa de entrega al destinatario queda encapsulada en cada proveedor.

2. Nivel 2 — Contenedores

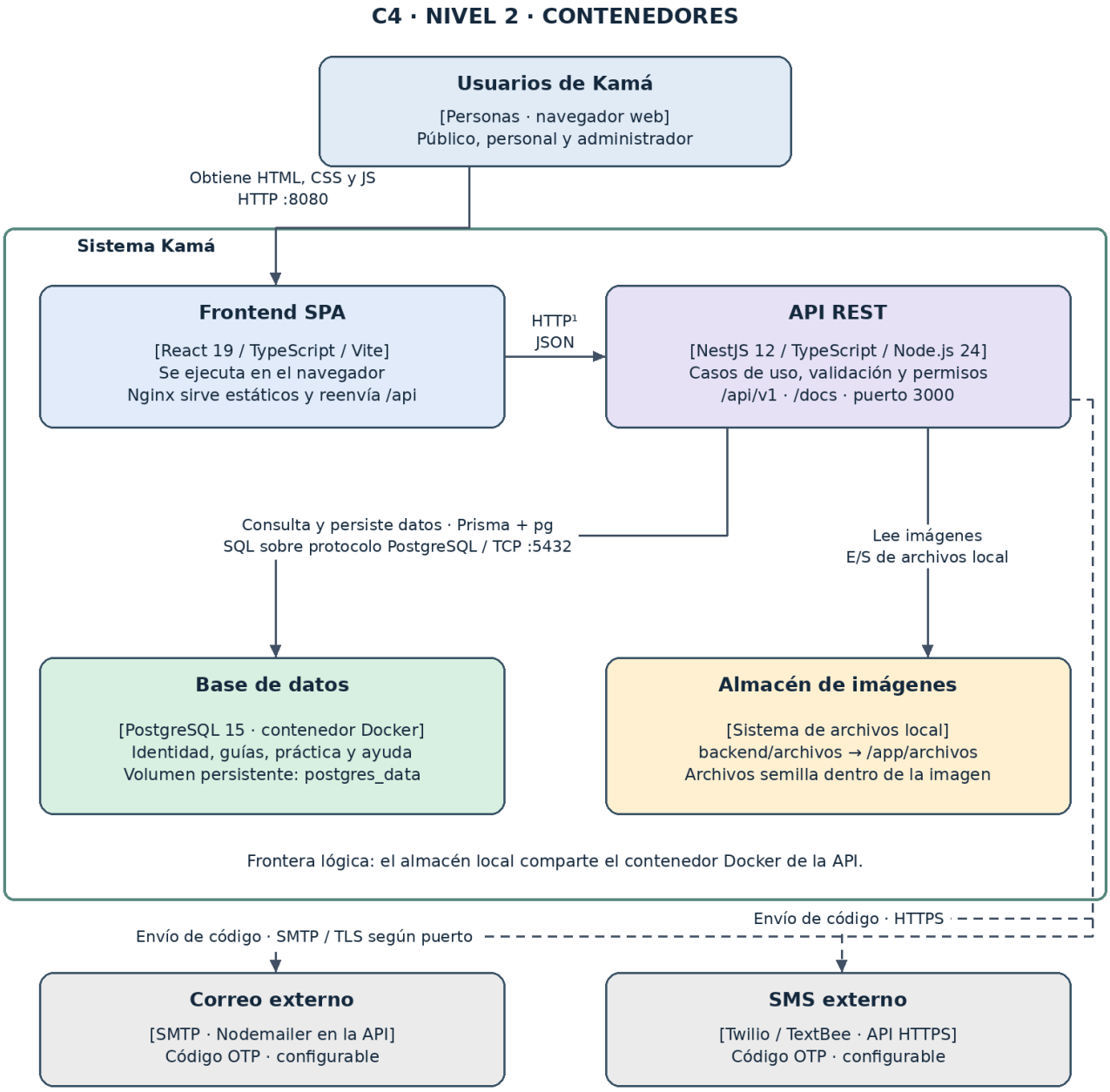


Figura 2. Unidades de ejecución, datos e integraciones.

Leyenda: azul: frontend; lila: API; verde: datos; ocre: archivos; gris: externos. Líneas continuas: comunicación local; discontinuas: proveedores configurables. Un contenedor C4 no equivale necesariamente a un contenedor Docker.

¹ **API:** el navegador solicita `/api/v1` mediante HTTP/JSON a Nginx (:8080); Nginx reenvía por HTTP a `backend:3000`. Las imágenes viajan por HTTP con su tipo MIME. HTTPS/JSON queda previsto para un despliegue con terminación TLS.

3. Comunicaciones y despliegue

Origen → destino	Contenido y protocolo	Estado
Navegador → frontend/Nginx	HTML, CSS, JavaScript; HTTP :8080	Compose local
SPA → API, vía Nginx	Peticiones y respuestas JSON; HTTP /api/v1; JWT y cookie de sesión cuando corresponda	Implementado
API → PostgreSQL	Lecturas y escrituras mediante Prisma y pg; SQL / protocolo PostgreSQL TCP :5432	Implementado
API → almacén local	Lectura de imágenes mediante sistema de archivos; entrega HTTP en /api/v1/archivos	Implementado
API → correo	Código de verificación por SMTP; TLS implícito en puerto 465; otros puertos según negociación del servidor	Adaptador configurable
API → SMS	Código de verificación por HTTPS a Twilio o TextBee	Adaptadores configurables

Arranque local. Docker Compose declara PostgreSQL 15, backend y frontend. PostgreSQL usa un volumen persistente y healthcheck; el frontend depende de que el backend esté sano. Nginx sirve la SPA y actúa como proxy de `/api/`. Swagger se consulta directamente en `http://localhost:3000/docs`; el proxy actual no declara una ruta específica para `/docs`.

Persistencia de archivos. Las imágenes semilla están en `backend/archivos` y se incluyen en la imagen del backend. No se declara un volumen de archivos en Compose ni un servicio de objetos. Cualquier futura carga persistente requiere definir almacenamiento y ciclo de vida; no debe confundirse la tabla de metadatos de imagen con el archivo binario.

Producción. Quedan por concretar terminación HTTPS, proveedor real y configuración de notificaciones, almacenamiento duradero de archivos y operación del despliegue. El diagrama no representa esos pendientes como infraestructura ya aprovisionada.

4. Nivel 3 — Componentes de la API

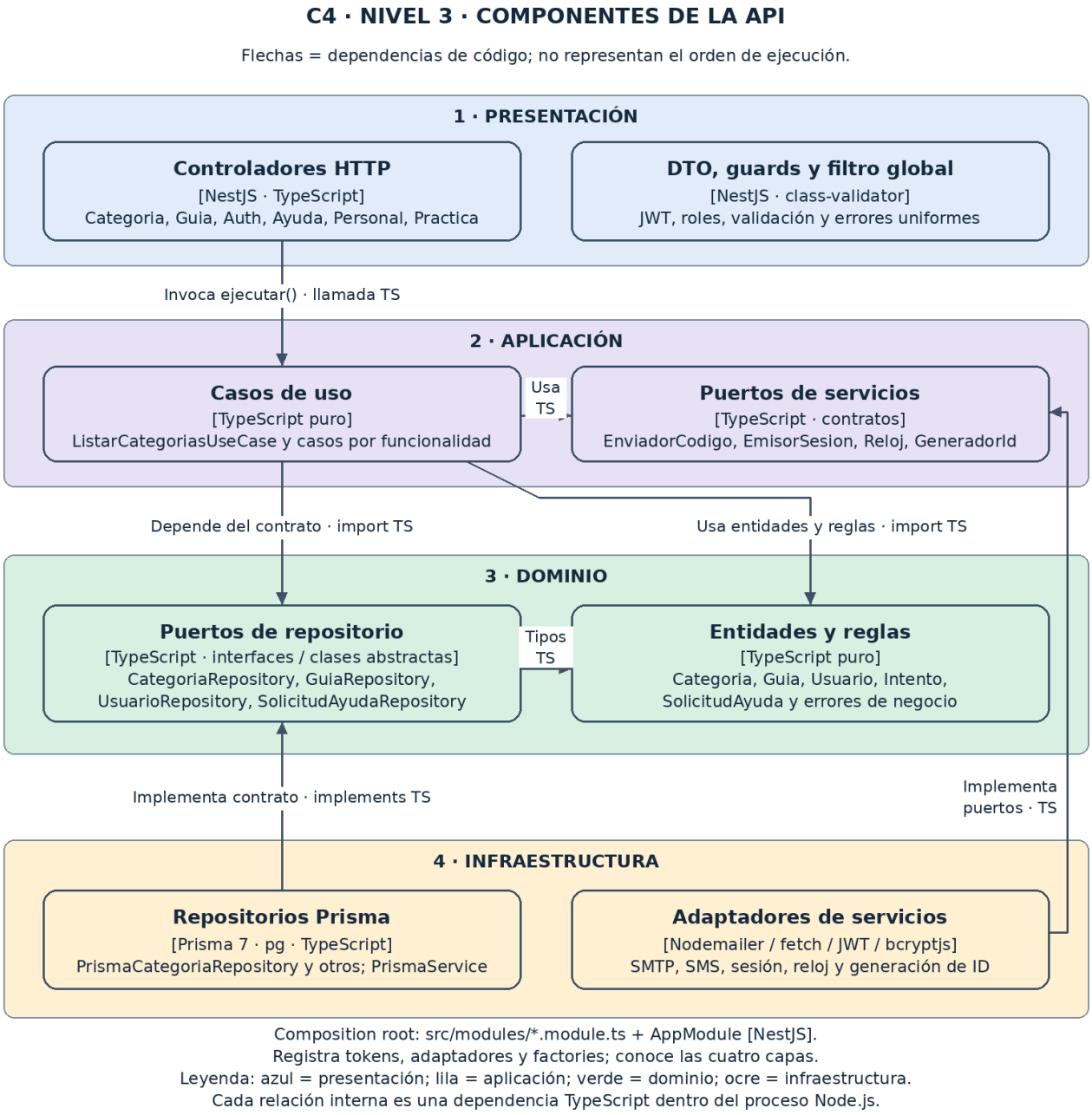


Figura 3. Dependencias agrupadas en las cuatro capas.

Vista resumida por responsabilidad. Las interfaces de repositorio pertenecen al dominio; los puertos para servicios técnicos pertenecen a aplicación. Los adaptadores de infraestructura apuntan a esos contratos internos. La composición NestJS queda fuera del núcleo y ensambla las dependencias.

5. Regla de dependencia

Presentación. Los controladores reciben peticiones, los DTO definen la forma válida de entrada y los guards aplican autenticación y permisos. El filtro global traduce excepciones a respuestas uniformes. Esta capa invoca casos de uso y puede referenciar tipos internos.

Aplicación. Los casos de uso coordinan acciones del negocio con repositorios y servicios expresados como contratos. No importan NestJS ni Prisma. Por ejemplo, `ListarCategoriasUseCase` recibe un `CategoriaRepository` en su constructor y genera un resultado paginado.

Dominio. Contiene entidades, reglas, errores de negocio y contratos de repositorio. No conoce HTTP, NestJS, PostgreSQL ni el cliente de Prisma. Esta independencia permite probar reglas con objetos TypeScript y reemplazar adaptadores.

Infraestructura. Implementa repositorios con Prisma, transforma registros en entidades y encapsula servicios de correo, SMS, seguridad, configuración y logging. `PrismaCategoriaRepository` implementa el contrato definido dentro del dominio; el caso de uso no importa esa implementación.

Ejemplo: listar categorías

1. El navegador solicita `GET /api/v1/categorias` con filtros y paginación.
2. `CategoriaController` recibe `ListarCategoriasDto`, validado por el pipe global, y llama a `ejecutar()`.
3. `ListarCategoriasUseCase` invoca `categorias.listar(criterio)` a través del contrato.
4. La instancia real es `PrismaCategoriaRepository`: consulta PostgreSQL mediante `PrismaService` y construye entidades `Categoria`.
5. El caso de uso devuelve datos, total, página y límite; el controlador entrega JSON.

Dependencia frente a ejecución. En ejecución se llama al adaptador concreto, pero la dependencia de código del caso de uso apunta al contrato interno. NestJS resuelve la implementación al arrancar; por eso el flujo de datos hacia PostgreSQL no invierte la regla de dependencia.

6. Estructura real del backend

```
backend/
├── src/
│   ├── domain/          Entidades, reglas y repositorios abstractos
│   │   └── auth, ayuda, categoria, guia, imagen, personal, practica, shared
│   ├── application/     Casos de uso, resultados y puertos
│   │   └── auth, ayuda, categoria, guia, personal, practica, shared
│   ├── presentation/    Controladores, DTO, guards y filtros
│   │   └── auth, ayuda, categoria, guia, personal, practica,
│   │       health, filters, shared
│   ├── infrastructure/  Adaptadores técnicos
│   │   ├── database/    PrismaService y repositories/
│   │   ├── notificaciones/  Correo, SMS, consola y plantillas
│   │   ├── seguridad/   JWT, bcrypt y códigos
│   │   ├── config/      Validación de variables de entorno
│   │   ├── logging/     Configuración de Pino
│   │   └── sistema.ts   Reloj e identificadores
│   ├── modules/         Composition root por funcionalidad
│   ├── app.module.ts    Ensamblaje global
│   ├── app.setup.ts     Prefijo, validación, CORS, archivos, Swagger
│   └── main.ts          Arranque del servidor
├── prisma/
│   ├── schema.prisma    Modelo de persistencia
│   ├── migrations/      Migraciones SQL versionadas
│   └── seed.ts          Datos semilla
├── archivos/semilla/guias/  Imágenes servidas por la API
├── test/e2e/            Pruebas HTTP de integración
├── prisma.config.ts
├── Dockerfile
└── .env.example
```

Árbol resumido: se omiten archivos repetidos, dependencias instaladas y código generado. Las pruebas unitarias se colocan en carpetas `__tests__` junto a la funcionalidad. El cliente Prisma se genera bajo `src/infrastructure/database/prisma/generated` según el esquema.

Dónde agregar una funcionalidad. Sus entidades y contratos van en `domain/<funcionalidad>`; la coordinación, en `application/`; endpoints y DTO, en `presentation/`; acceso a datos, en `infrastructure/database/repositories/`. El módulo registra las implementaciones y fábricas, sin mover reglas de negocio al módulo.

7. Decisiones técnicas justificadas

Decisión implementada	Justificación arquitectónica y compromiso
ORM: Prisma 7 + adapter-pg / pg	El esquema y las migraciones versionadas mantienen explícita la estructura de PostgreSQL. Los repositorios encapsulan consultas y mapean registros a entidades, evitando filtrar tipos del ORM al núcleo. Compromiso: mantener los mapeos y generar el cliente en el build.
Validación: class-validator + class-transformer	Se integran con los DTO y ValidationPipe global de NestJS. whitelist y forbidNonWhitelisted rechazan propiedades inesperadas; transform convierte la entrada a DTO. Los errores se normalizan en español. Las invariantes de negocio permanecen en dominio.
Logger: nestjs-pino + pino-http	Centraliza registros HTTP estructurados y niveles por entorno. Oculta cabeceras Authorization, Cookie y Set-Cookie; omite healthchecks rutinarios. pino-pretty mejora lectura en desarrollo. Compromiso: revisar redacción de cualquier campo sensible añadido.
Frontend: componentes propios React + Tailwind CSS 4	El package.json no declara MUI, Ant Design, shadcn/ui u otra biblioteca externa de componentes. Se reutilizan componentes locales como SiteHeader, AccessibilityBar y AvisoPrivacidadModal, con estilos y tokens propios. Tailwind es una herramienta de estilos, no una biblioteca de componentes. Permite adaptar la interfaz al público del CLENA; exige verificar accesibilidad y consistencia por componente.
Monolito modular NestJS	Concentra módulos por funcionalidad en un único proceso, reduce complejidad operativa para el proyecto y mantiene separación por capas. Los contratos permiten sustituir infraestructura; no se requiere desplegar cada caso de uso como servicio independiente.

Las versiones mayores se toman de los manifiestos del repositorio; no son una recomendación de actualización. Las decisiones describen la implementación observada. No se atribuye una aprobación formal a un acta no disponible.

8. Inyección por token en NestJS

Fragmento real de `backend/src/modules/categoria.module.ts`. La clase abstracta `CategoriaRepository` existe en ejecución y funciona como token de inyección; no se usa una interfaz TypeScript borrada al compilar.

```
import { Module } from '@nestjs/common';
import { ListarCategoriasUseCase } from '../application/categoria/listar-categorias.use-
case.js';
import { CategoriaRepository } from '../domain/categoria/categoria.repository.js';
import { PrismaCategoriaRepository } from '../infrastructure/database/repositories/prisma-
categoria.repository.js';
import { CategoriaController } from '../presentation/categoria/categoria.controller.js';

@Module({
  controllers: [CategoriaController],
  providers: [
    { provide: CategoriaRepository, useClass: PrismaCategoriaRepository },
    {
      provide: ListarCategoriasUseCase,
      useFactory: (categorias: CategoriaRepository) =>
        new ListarCategoriasUseCase(categorias),
      inject: [CategoriaRepository],
    },
  ],
})
export class CategoriaModule {}
```

`provide/useClass` enlaza el contrato con `PrismaCategoriaRepository`. `inject` solicita ese token y `useFactory` construye el caso de uso con el repositorio resuelto. El caso de uso no necesita `@Injectable()` ni importar NestJS.

```
// backend/src/application/categoria/listar-categorias.use-case.ts
export class ListarCategoriasUseCase {
  constructor(private readonly categorias: CategoriaRepository) {}
  // ejecutar(criterio) llama a this.categorias.listar(criterio)
}
```

El segundo fragmento abrevia el cuerpo para resaltar el constructor. `AppModule` importa `PrismaModule` y `CategoriaModule`; el primero expone `PrismaService` globalmente. Así el adaptador recibe el cliente de persistencia sin acoplar dominio o aplicación al contenedor de NestJS.

9. Trazabilidad y límites

Requisito E5	Evidencia del informe
Nivel 1: sistema, actores y externos	Figura 1, página 3
Nivel 2: frontend, API, BD, archivos y correo	Figura 2, página 4; protocolos en página 5
Nivel 3: cuatro capas y dependencias internas	Figura 3, página 6; explicación en página 7
Estructura real de carpetas	Página 8
ORM, validación, logger y componentes frontend	Página 9
Módulo NestJS con inyección de repositorio	Página 10
Formato: portada, índice, páginas e imágenes	Portada e índice; figuras PNG incrustadas y numeradas

Fuentes locales verificables

`Entrega 1.pdf` (enunciado facilitado); `docker-compose.yml`; manifiestos y Dockerfiles de ambos proyectos; `frontend/nginx.conf`; `backend/src/app.module.ts`, `app.setup.ts`, `modules/categoria.module.ts` y `modules/auth.module.ts`; capas de categoría; `infrastructure/logging/logging.module.ts`; `presentation/shared/pipe-validacion.ts`; `prisma/schema.prisma` y migraciones.

Los nombres de integrantes proceden de `docs/LEER_PRIMERO_Validacion_ODD_Kama.md`. Se conservan los nombres disponibles y no se inventan apellidos o validaciones del equipo.

Límites y revisión antes de entregar

Este E5 documenta arquitectura, no certifica cumplimiento completo de E6/E7, despliegue productivo ni pruebas funcionales de todo el producto. El proveedor de correo/SMS y HTTPS requieren configuración de despliegue. La elección actual de componentes propios se declara explícitamente para que el equipo pueda justificarla ante el requisito de biblioteca de componentes del frontend.

El enunciado exige que los entregables estén en **main mediante Pull Request**. La generación de este informe no constituye un commit, PR ni fusión. El equipo debe revisar datos de portada y exponer las decisiones durante la presentación. Las imágenes se conservan en `docs/img/` y el material regenerable en `docs/arquitectura/`.